

JavaScript

Table of Contents

arrays

async-await

browser-apis

callbacks

closures

control-flow

event-loop

functions

fundamentals

modules

objects

operators

promises

prototypes-classes

sets-maps-symbols

strings

Arrays

An array is a data structure that stores an ordered collection of values, called elements. These elements can be of any data type, such as numbers, strings, objects, or even other arrays.

JS

```
const mixedArray = ["Hello", 10, true, null, { name: 'Juan',  
job: 'Programmer' }, [1, 2, 3]];
```

Creating Arrays

a) Using Bracket Notation

JS

```
let fruits = ["apple", "banana", "orange"];
```

b) Using the Array Constructor

JS

```
let fruits = new Array("apple", "banana", "orange");
```

Accessing Elements

You can access array elements using numeric indexes. In JavaScript, indexes start at 0, meaning the first element has index 0, the second has index 1, and so on.

JS

```
let fruits = ["apple", "banana", "orange"];  
console.log(fruits[0]); // "apple"  
console.log(fruits[2]); // "orange"
```

Modifying Elements

You can modify array elements by accessing them through their index and assigning a new value.

JS

```
fruits[1] = "pear";  
console.log(fruits); // ["apple", "pear", "orange"]
```

Instance Methods

- length

JS

```
let numbers = [1, 2, 3, 4];  
console.log(numbers.length); // 4
```

- push()

JS

```
let numbers = [1, 2, 3, 4];  
numbers.push(5);  
console.log(numbers); // [1, 2, 3, 4, 5]
```

- pop()

JS

```
let numbers = [1, 2, 3, 4, 5];  
let last = numbers.pop();  
console.log(last); // 5  
console.log(numbers); // [1, 2, 3, 4]
```

- shift()

JS

```
let numbers = [1, 2, 3, 4];  
let first = numbers.shift();  
console.log(first); // 1  
console.log(numbers); // [2, 3, 4]
```

- unshift()

JS

```
let numbers = [2, 3, 4];  
numbers.unshift(0);  
console.log(numbers); // [0, 2, 3, 4]
```

- `indexOf()`

JS

```
let numbers = [0, 2, 3, 4];
console.log(numbers.indexOf(3)); // 2
console.log(numbers.indexOf(10)); // -1
```

- `slice()`

JS

```
let numbers = [0, 2, 3, 4];
let subArray = numbers.slice(1, 3);
console.log(subArray); // [2, 3]
```

- `splice()` The `splice()` method is used to add, remove, or replace elements in an array.

JS

```
array.splice(start, deleteCount, item1, item2, ..., itemN)
```

- **start** (required) – The index where the operation starts:
 - A positive value counts from the beginning (0 is the first element).
 - A negative value counts from the end (-1 is the last element).
- **deleteCount** (optional) – The number of elements to remove:
 - If 0, no elements are removed.
 - If omitted, all elements from start to the end are removed.
- **item1, item2, ..., itemN** (optional) – New elements to insert at start:
 - If omitted, `splice()` only removes elements (if `deleteCount > 0`).

Return Value:

- Returns a new array containing the removed elements.
- If no elements are removed, an empty array (`[]`) is returned.

Removing Elements from an Array

JS

```
let fruits = ['apple', 'banana', 'cherry', 'date'];
let removed = fruits.splice(1, 2); // Removes 2 elements starting from index 1
console.log(fruits); // ['apple', 'date']
console.log(removed); // ['banana', 'cherry']
```

Adding Elements to an Array

JS

```
let colors = ['red', 'blue', 'green'];
colors.splice(1, 0, 'yellow', 'orange'); // Inserts without removing
console.log(colors); // ['red', 'yellow', 'orange', 'blue', 'green']
```

Replacing Elements in an Array

JS

```
let numbers = [1, 2, 3, 4];
let replaced = numbers.splice(1, 2, 9, 8); // Replaces 2 elements at index 1
console.log(numbers); // [1, 9, 8, 4]
console.log(replaced); // [2, 3]
```

Removing Elements from a Specific Index to the End

JS

```
let animals = ['cat', 'dog', 'elephant', 'giraffe'];
let removed = animals.splice(2); // Removes all elements from index 2 onwards
console.log(animals); // ['cat', 'dog']
console.log(removed); // ['elephant', 'giraffe']
```

Using Negative Indexes

JS

```
let cities = ['London', 'Paris', 'Berlin', 'Tokyo'];
cities.splice(-2, 1); // Removes 1 element at index -2 (Berlin)
console.log(cities); // ['London', 'Paris', 'Tokyo']
```

- `concat()`

JS

```
let numbers = [0, 2, 3, 4];
let otherNumbers = [5, 6];
let combined = numbers.concat(otherNumbers);
console.log(combined); // [0, 2, 3, 4, 5, 6]
```

- `join()`

JS

```
let numbers = [0, 2, 3, 4];
let str = numbers.join("-");
console.log(str); // "0-2-3-4"
```

- reverse()

JS

```
let numbers = [0, 2, 3, 4];
numbers.reverse();
console.log(numbers); // [4, 3, 2, 0]
```

- sort()

JS

```
let letters = ["d", "a", "c", "b"];
letters.sort();
console.log(letters); // ["a", "b", "c", "d"]

let nums = [10, 5, 100, 1];
nums.sort((a, b) => a - b);
console.log(nums); // [1, 5, 10, 100]
```

Caution when sorting numbers! Since numbers are converted to strings, sorting `[10, 5, 100, 1]` without a comparator function results in:

JS

```
let nums = [10, 5, 100, 1];
nums.sort();
console.log(nums); // [1, 10, 100, 5] - wrong order
```

Functional Methods

- map() – Transform Elements

JS

```
let numbers = [4, 3, 2, 0];
let doubles = numbers.map(num => num * 2);
console.log(doubles); // [8, 6, 4, 0]
```

- filter() – Select Elements

JS

```
let numbers = [4, 3, 2, 0];
let greaterThanTwo = numbers.filter(num => num > 2);
console.log(greaterThanTwo); // [4, 3]
```

- reduce() – Accumulate a Value

JS

```
let numbers = [4, 3, 2, 0];
let sum = numbers.reduce((accumulator, num) => accumulator + num, 0);
console.log(sum); // 9
```

- find() – Find the First Match

JS

```
let firstGreaterThanTwo = numbers.find(num => num > 2);
console.log(firstGreaterThanTwo); // 4
```

- some() – Check for At Least One Match

JS

```
let hasNegatives = numbers.some(num => num < 0);
console.log(hasNegatives); // false
```

- every() – Check If All Elements Match

JS

```
let allPositive = numbers.every(num => num > 0);
console.log(allPositive); // false
```

Multidimensional Arrays

JS

```
let matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];
console.log(matrix[0][1]); // 2 (row 0, column 1)
console.log(matrix[2][2]); // 9 (row 2, column 2)
```

Iterating Over Arrays

- for loop:

JS

```
let numbers = [1, 2, 3, 4];
for (let i = 0; i < numbers.length; i++) {
  console.log(numbers[i]);
}
```

- for...of loop:

JS

```
for (let number of numbers) {
  console.log(number);
}
```

- forEach():

JS

```
numbers.forEach(function (number) {
  console.log(number);
});
numbers.forEach(number => console.log(number));
```

Iterating Over Multidimensional Arrays

JS

```
let matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];
for (let row of matrix) {
  for (let element of row) {
    console.log(element);
  }
}
/* 1 2 3
   4 5 6
   7 8 9 */
```

Async / Await

Async/Await diagram

JavaScript async Keyword We use the async keyword with a function to represent that the function is an asynchronous function. The async function returns a promise.

JavaScript await Keyword The await keyword is used only inside the async function to wait for the asynchronous operation. **Benefits of Using async Function**

- The code is more readable than using a callback or a promise.
- Error handling is simpler.
- Debugging is easier.

JS

```
async function obtenerDatos() {
  try {
    const response = await
    fetch('https://jsonplaceholder.typicode.com/posts/1');
    if (!response.ok) {
      throw new Error('Error en la solicitud');
    }
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error('Hubo un problema:', error);
  }
}
obtenerDatos();
```

Browser APIs & Storage

Window

`window` is a global object that represents the browser window where the JavaScript code runs in a browser environment.

Key Characteristics of `window` :

- **Global Object:** Contains properties and methods that can be used anywhere in your code.
- **Access to DOM Elements:** Provides access to everything related to the browser's user interface.

Common Methods and Properties:

- `window.alert()` - Displays an alert box in the browser.
- `window.confirm()` - Displays a confirmation dialog box.
- `window.prompt()` - Displays a dialog box asking the user for input.
- `window.setTimeout()` - Executes a function after a specified time interval.
- `window.setInterval()` - Repeatedly executes a function with a specified time interval.
- `window.clearTimeout()` and `window.clearInterval()` - Cancel timeouts or intervals set with `setTimeout()` and `setInterval()`.
- `window.location` - Contains information about the current URL and allows changing the URL.
- `window.document` - Provides access to the DOM of the current page.
- `window.localStorage` and `window.sessionStorage` - Provide data storage in the browser.
- `window.innerWidth` and `window.innerHeight` - Return the dimensions of the browser window.
- `window.console` - Provides access to the console for debugging (e.g., `console.log()`).

Common Window Events:

- `resize` - Fired when the window is resized.
- `scroll` - Fired when the user scrolls the window's content.
- `load` - Fired when the window and all its resources are fully loaded.

Self

En el contexto de los navegadores web, `self` es una referencia global al objeto global de la ventana (el objeto `window`). `self` y `window` son equivalentes en los navegadores.

JS

```
self.alert("Hello!"); // Es equivalente a window.alert("Hello!")
console.log(self === window); // true
self.console.log("Using self!"); // Funciona igual que window.console.log
```

Global

In a browser environment, the global object is `window`. In Node.js, the global object is called `global`.

Key Methods and Properties in Node.js:

- `global.setTimeout()` / `global.clearTimeout()`
- `global.setInterval()` / `global.clearInterval()`
- `global.process` - Provides information and control over the current Node.js process.
- `global.console` - Provides access to the standard output and error streams.
- `global.Buffer` - Represents binary data buffers.
- `global.require` - Function to load modules.

Fetch API

Es una interfaz moderna que permite realizar solicitudes HTTP (GET, POST, PUT, DELETE) de manera asíncrona desde el navegador hacia un servidor.

Basada en Promesas: `fetch` devuelve una promesa que se resuelve con la respuesta de la solicitud.

Manejo de CORS: `fetch` respeta las políticas de CORS.

JS

```
fetch('https://jsonplaceholder.typicode.com/posts/1')
  .then(response => {
    if (!response.ok) {
      throw new Error('Error en la solicitud: ' + response.statusText);
    }
    return response.json();
  })
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.error('Hubo un problema con la solicitud:', error);
  });
```

Opciones de Configuración en fetch

El segundo argumento opcional de `fetch` es un objeto de configuración:

- `method` : El método HTTP (GET, POST, PUT, DELETE).
- `headers` : Objeto con los encabezados HTTP.
- `body` : El cuerpo de la solicitud.
- `mode` : 'cors', 'no-cors', o 'same-origin'.
- `credentials` : 'omit', 'same-origin', o 'include'.
- `cache` : 'default', 'no-store', 'reload', 'no-cache', 'force-cache', o 'only-if-cached'.

JS

```
fetch('https://jsonplaceholder.typicode.com/posts/1', {
  method: 'GET',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': 'Bearer token'
  }
})
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error('Error:', error));
```

Cookies

Las cookies son pequeños fragmentos de datos almacenados por el navegador en el equipo del usuario.

Estructura de una cookie

TEXT

```
name=value; Expires=date; Max-Age=seconds; Domain=domain; Path=path; Secu
```

JS

```
document.cookie = "sessionID=abcd1234; Domain=example.com; Path=/users; E
```

Crear y establecer una cookie

JS

```
document.cookie = "username=JohnDoe; Max-Age=3600; Path=/; Secure; SameSi
```

Leer cookies

JS

```
console.log(document.cookie);  
// "username=JohnDoe; theme=dark"
```

Función para acceder a una cookie específica

JS

```
function getCookie(name) {  
    let cookies = document.cookie.split(';');  
    for (let cookie of cookies) {  
        let [key, value] = cookie.trim().split('=');  
        if (key === name) {  
            return decodeURIComponent(value);  
        }  
    }  
    return null;  
}  
  
console.log(getCookie("username")); // "JohnDoe"
```

Actualizar una cookie

JS

```
document.cookie = "username=JaneDoe; Max-Age=7200; Path=/";
```

Eliminar una cookie

JS

```
document.cookie = "Username=; Expires=Thu, 01 Jan 1970 00:00:00 UTC; Path
```

Configuración entre dominios (CORS)

Backend:

JS

```
app.get('/set-cookie', (req, res) => {
  res.cookie('sessionID', 'abcd1234', {
    httpOnly: true,
    secure: true,
    sameSite: 'None',
    domain: 'example.com',
    path: '/',
    maxAge: 24 * 60 * 60 * 1000,
  });
  res.status(200).send('Cookie enviada');
});
```

Frontend:

JS

```
fetch('https://api.example.com/set-cookie', {
  method: 'GET',
  credentials: 'include',
})
  .then((response) => response.text())
  .then((data) => console.log(data))
  .catch((error) => console.error('Error:', error));
```

Configuración CORS en el backend:

JS

```
const cors = require('cors');
app.use(
  cors({
    origin: 'https://frontend.com',
    credentials: true,
  })
);
```

Data Storage

Modern storage APIs like Web Storage API (`localStorage` and `sessionStorage`) and IndexedDB are recommended over cookies for client-side data storage.

Local Storage

Key Characteristics:

- **Capacity:** 5–10 MB per domain.
- **Persistence:** Data does not expire.
- **Synchronous:** All operations are synchronous.
- **String-based:** Only stores strings.

JS

```
// Storing data
localStorage.setItem('username', 'JohnDoe');
localStorage.setItem('theme', 'dark');

// Retrieving data
const username = localStorage.getItem('username'); // "JohnDoe"
console.log(username);

// Updating data
localStorage.setItem('username', 'JaneDoe');

// Deleting specific data
localStorage.removeItem('username');

// Clearing all data
localStorage.clear();
```

Storing Non-String Data:

JS

```
const user = { name: 'John', age: 30 };
localStorage.setItem('user', JSON.stringify(user));

const userData = JSON.parse(localStorage.getItem('user'));
console.log(userData.name); // "John"
```

Session Storage

Key Characteristics:

- **Scope:** Data is specific to a single browser tab or window.
- **Persistence:** Data persists only until the tab or window is closed.

- **Capacity:** 5–10 MB per domain.
- **String-based:** Only stores strings.

JS

```
// Storing data
sessionStorage.setItem('username', 'JohnDoe');
sessionStorage.setItem('theme', 'dark');

// Retrieving data
const username = sessionStorage.getItem('username'); // "JohnDoe"
console.log(username);

// Updating data
sessionStorage.setItem('username', 'JaneDoe');

// Deleting specific data
sessionStorage.removeItem('username');

// Clearing all data
sessionStorage.clear();
```

Storing Non-String Data:

JS

```
const user = { name: 'John', age: 30 };
sessionStorage.setItem('user', JSON.stringify(user));

const userData = JSON.parse(sessionStorage.getItem('user'));
console.log(userData.name); // "John"
```

Best Practices

- Avoid storing sensitive data (passwords, tokens) in Storage APIs.
- Check browser availability:

JS

```
if (typeof sessionStorage !== 'undefined') {
    console.log('Session Storage is available');
}
```

IndexedDB

IndexedDB is a low-level API for client-side storage of significant amounts of structured data, including files and blobs.

Key Features:

- **Asynchronous:** Doesn't block the main thread.
- **Persistent:** Data persists after browser is closed.
- **Indexed:** Supports efficient searches using indexes.
- **Large Storage:** Can store large amounts of data.
- **Transactions:** All operations are part of a transaction.

Opening or Creating a Database

JS

```
const request = indexedDB.open('MyDatabase', 1);

request.onupgradeneeded = (event) => {
  const db = event.target.result;
  const objectStore = db.createObjectStore('users', { keyPath: 'id' });
  objectStore.createIndex('name', 'name', { unique: false });
};

request.onsuccess = (event) => {
  const db = event.target.result;
  console.log('Database opened:', db);
};

request.onerror = (event) => {
  console.error('Database error:', event.target.error);
};
```

Adding Data

JS

```
const db = request.result;
const transaction = db.transaction('users', 'readwrite');
const objectStore = transaction.objectStore('users');

objectStore.add({ id: 1, name: 'John Doe', age: 30 });
objectStore.add({ id: 2, name: 'Jane Doe', age: 25 });

transaction.oncomplete = () => console.log('Transaction complete');
transaction.onerror = (event) => console.error('Transaction error:', event);
```

Reading Data

JS

```
const transaction = db.transaction('users', 'readonly');
const objectStore = transaction.objectStore('users');

const request = objectStore.get(1);
request.onsuccess = () => console.log('Data:', request.result);

const index = objectStore.index('name');
const query = index.get('Jane Doe');
query.onsuccess = () => console.log('Query Result:', query.result);
```

Updating Data

JS

```
const transaction = db.transaction('users', 'readwrite');
const objectStore = transaction.objectStore('users');

objectStore.put({ id: 1, name: 'John Smith', age: 35 });

transaction.oncomplete = () => console.log('Update complete');
```

Deleting Data

JS

```
const transaction = db.transaction('users', 'readwrite');
const objectStore = transaction.objectStore('users');

objectStore.delete(1);
objectStore.clear();

transaction.oncomplete = () => console.log('Delete complete');
```

Deleting a Database

JS

```
const deleteRequest = indexedDB.deleteDatabase('MyDatabase');
deleteRequest.onsuccess = () => console.log('Database deleted');
deleteRequest.onerror = (event) => console.error('Delete error:', event.target.error);
```

DOM (Document Object Model)



DOM Tree structure

It is a programming interface that allows us to modify elements from the document. The HTML DOM is a standard for how to get, change, add, or delete HTML elements.

Accessing DOM Elements

JS

```
const headerElement = document.getElementById('header');
const paragraphs = document.getElementsByClassName('paragraph');
const images = document.getElementsByTagName('img');
```

Modifying Element Content

JS

```
headerElement.innerHTML = 'New Header Text';
```

Events and Event Handling

JS

```
const myButton = document.getElementById('myButton');
myButton.addEventListener('click', function () {
    alert('Button Clicked!');
});
```

Manipulating Styles

JS

```
const myParagraph = document.getElementById('myParagraph');
const colorButton = document.getElementById('colorButton');

colorButton.addEventListener('click', function () {
    myParagraph.style.color = 'blue';
});
```

Creating New Elements

JS

```
const newParagraph = document.createElement('p');
newParagraph.textContent = 'This is a new paragraph.';
document.body.appendChild(newParagraph);
```

Modifying Attributes

JS

```
const myImage = document.getElementById('myImage');
myImage.src = 'new-image.jpg';
```

Updating Form Input Values

JS

```
const myInput = document.getElementById('myInput');
myInput.addEventListener('input', function () {
    document.getElementById('inputValue').textContent = myInput.value;
});
```

Toggling Visibility

JS

```
const toggleButton = document.getElementById('toggleButton');
const toggleParagraph = document.getElementById('toggleParagraph');

toggleButton.addEventListener('click', function () {
    toggleParagraph.style.display = toggleParagraph.style.display === 'none' ? 'block' : 'none';
});
```

DOMContentLoaded

JS

```
document.addEventListener('DOMContentLoaded', function () {
    // DOM manipulation code goes here
});
```

Checking if an Element Exists

JS

```
const myElement = document.getElementById('myElement');
if (myElement) {
    // Manipulate the element here
} else {
    console.error('Element not found!');
}
```

Preventing Default Actions

JS

```
const myForm = document.getElementById('myForm');
myForm.addEventListener('submit', function (event) {
  event.preventDefault();
  // Your form handling code goes here
});
```

JavaScript Callback Function

A callback is a function that is passed as an argument to another function and is executed after some operation has been completed. Callbacks are often used for handling asynchronous operations, such as reading files, making API requests, or handling events like user input.

Synchronous Callback

JS

```
function greet(name, callback) {  
    console.log("Hello, " + name + "!");  
    callback();  
}  
  
function sayGoodbye() {  
    console.log("Goodbye!");  
}  
  
greet("Alice", sayGoodbye);
```

Asynchronous Callback

This example shows an asynchronous operation using `setTimeout`, where the callback is executed after a delay.

JS

```
function fetchData(callback) {  
    console.log("Fetching data...");  
    setTimeout(() => {  
        console.log("Data fetched.");  
        callback();  
    }, 2000);  
}  
  
function processData() {  
    console.log("Processing data...");  
}  
  
fetchData(processData);
```

Output:

TEXT

```
Fetching data...  
Data fetched.  
Processing data...
```

Callback Hell



Callback Hell

Callback Hell es un término que describe una situación en la programación, especialmente en JavaScript, donde tienes múltiples funciones asíncronas que dependen unas de otras. Cuando estas funciones se manejan usando callbacks, el código puede volverse anidado y difícil de leer y mantener. Este anidamiento excesivo se conoce como "Callback Hell" o "Pyramid of Doom".

Closures

Closures in JavaScript are functions that "remember" the scope in which they were created, even after that scope has finished executing. In other words, a closure allows a function to access variables from an outer function even after the outer function has returned.

How Closures Work

When a function is created inside another function, it has access to:

- Its own local variables.
- The variables of the outer function.
- Global variables.

Even if the outer function finishes execution, the inner function can still access the outer function's variables because they are kept in memory.

JS

```
function outerFunction(outerVariable) {  
    return function innerFunction(innerVariable) {  
        console.log(`Outer: ${outerVariable}, Inner: ${innerVariable}`);  
    };  
}  
  
const closureInstance = outerFunction("Hello");  
closureInstance("World"); // Output: Outer: Hello, Inner: World
```

JS

```
function makeFunc() {  
    let name = 'Mozilla';  
    function displayName() {  
        alert(name);  
    }  
    return displayName;  
}  
  
let myFunc = makeFunc();  
myFunc();
```


JS

```
function makeCounter() {  
  let count = 0;  
  return function () {  
    count++;  
    return count;  
  };  
}  
  
let counter = makeCounter();  
console.log(counter()); // 1  
console.log(counter()); // 2  
console.log(counter()); // 3
```

Control Flow

Conditional Structures

Conditional structures allow executing different code blocks based on certain conditions.

if

Executes a block of code if a condition is true.

JS

```
const age = 18;
if (age ≥ 18) {
  console.log("You are of legal age.");
}
```

if...else

Executes one block if the condition is true, and another block if the condition is false.

JS

```
const age = 16;
if (age ≥ 18) {
  console.log("You are of legal age.");
} else {
  console.log("You are underage.");
}
```

if...else if...else

Allows checking multiple conditions in sequence.

JS

```
const score = 85;
if (score ≥ 90) {
  console.log("Excellent");
} else if (score ≥ 80) {
  console.log("Very good");
} else {
  console.log("Needs improvement");
}
```

switch

Allows selecting one among multiple code blocks to execute based on the value of an expression.

JS

```
const day = 2;

switch (day) {
  case 1:
    console.log("Monday");
    break;
  case 2:
    console.log("Tuesday");
    break;
  case 3:
    console.log("Wednesday");
    break;
  default:
    console.log("Other day");
}
```

Loop Structures

Loop structures allow repeating a block of code multiple times.

for

Repeats a block of code a specific number of times.

JS

```
for (let i = 0; i < 5; i++) {
  console.log(i);
}
```

while

Repeats a block of code while a condition is true.

JS

```
let i = 0;
while (i < 5) {
  console.log(i);
  i++;
}
```

do...while

Similar to while, but guarantees the code block executes at least once, since the condition is checked after execution.

JS

```
let i = 0;

do {
  console.log(i);
  i++;
} while (i < 5);
```

Iterators

for...in

The for...in loop iterates over the enumerable properties of an object, including inherited properties through the prototype chain.

JS

```
const person = {
  name: 'Juan',
  age: 30,
  city: 'Madrid'
};
for (const key in person) {
  console.log(key, person[key]);
}
// name Juan
// age 30
// city Madrid
```

Notes:

- for...in iterates over object properties.
- for...in on arrays returns the indices.

JS

```
const tasks = ['Task', 'Eat', 'Project', 'Study JavaScript'];
for (let task in tasks) {
  console.log(task);
}
// 0
// 1
// 2
// 3
```

for...of

The for...of loop iterates over the elements of iterable objects, such as arrays, strings, maps, sets, etc. Unlike for...in, for...of does not iterate over object properties, but over the values of an iterable collection.

JS

```
const tasks = ['Task', 'Eat', 'Project', 'Study JavaScript'];
for (let task of tasks) {
  console.log(task);
}
// Task
// Eat
// Project
// Study JavaScript
```

Notes: for...of iterates over arrays (returns values).

JS

```
const person = {
  name: 'Juan',
  age: 30,
  city: 'Madrid'
};
// Object.entries(person) returns an array
for (const [key, value] of Object.entries(person)) {
  console.log(key, value);
}
// name Juan
// age 30
// city Madrid
```

map()

The `map()` method is an array function that creates a new array with the results of calling a provided function on every element in the original array.

JS

```
const numbers = [1, 2, 3, 4];
const doubles = numbers.map(num => num * 2);
console.log(doubles); // [2, 4, 6, 8]
```

Notes: `map()` does not modify the original array; it returns a new array with the transformed elements.

forEach()

The `forEach()` method executes a provided function once for each array element.

JS

```
const fruits = ['apple', 'banana', 'orange'];
fruits.forEach((fruit, index) => {
  console.log(index, fruit);
});
// 0 apple
// 1 banana
// 2 orange
```

Notes:

- `forEach()` does not return a new array; it simply executes the provided function for each element.

Event Loop & Async Patterns

Código Síncrono En un entorno síncrono, las operaciones se ejecutan una tras otra, en el orden en que están escritas. Esto significa que cada línea de código debe completarse antes de que la siguiente pueda comenzar.

JS

```
console.log('Inicio');
function tareaPesada() {
  // Simulando una tarea que toma tiempo
  for (let i = 0; i < 1e9; i++) {}
  console.log('Tarea pesada completada');
}
tareaPesada();
console.log('Fin');
```

Resultado: Inicio Tarea pesada completada Fin Características del Código Síncrono:

- Bloqueante: Si una operación tarda mucho, bloquea la ejecución del resto del código.
- Predecible: El flujo de ejecución es lineal y fácil de seguir.

Código Asíncrono En un entorno asíncrono, algunas operaciones pueden iniciarse, y mientras están en progreso, otras operaciones pueden ejecutarse. Esto es especialmente útil para operaciones que pueden tardar en completarse, como solicitudes de red, temporizadores, o leer archivos.

JS

```
console.log('Inicio');
setTimeout(() => {
  console.log('Tarea asíncrona completada');
}, 2000); // 2 segundos
console.log('Fin');
```

Resultado: Inicio Fin Tarea asíncrona completada Características del Código Asíncrono:

- No bloqueante: Las operaciones asíncronas permiten que el código continúe ejecutándose mientras se espera a que se completen.
- Impredecible: El orden de ejecución no es siempre secuencial, ya que depende de cuándo se completen las tareas asíncronas.

Event Loop The Event Loop is the mechanism that allows JavaScript to handle asynchronous, non-blocking operations despite being a single-threaded language. It ensures that code execution, event handling, and asynchronous tasks are coordinated efficiently through a task queue and other components.



Event Loop diagram

Key Components of the Event Loop

1. Call Stack:

- A LIFO (Last In, First Out) structure where functions are executed.
- When a function is invoked, it gets pushed onto the Call Stack.
- When the function completes execution, it is popped off the stack.

2. Web APIs (macrotasks):

- These are features provided by the environment (browser or Node.js) such as `setTimeout`, `fetch`, DOM events, etc.
- Asynchronous operations are delegated to these APIs, not directly executed in the Call Stack.

3. Task Queue (also called Callback Queue or Macrotask Queue):

- A queue that holds tasks (callbacks) ready to run after the Call Stack is empty.
- Examples of tasks: `setTimeout` or `setInterval` callbacks.

Nota: Task Queue = Callback Queue = Macrotask Queue. Los tres nombres se refieren a la misma cola. Se diferencia de la **Microtask Queue** (Promises, `queueMicrotask`) que tiene prioridad y se vacía antes de pasar a la Task Queue.

4. Microtask Queue:

- A queue with higher priority than the Task Queue.
- Examples: resolved `Promise.then` handlers, `MutationObserver` call backs.
- Microtasks are processed after the current code execution and before moving to the Task Queue.

5. Event Loop:

- Continuously monitors the Call Stack and queues.
- If the Call Stack is empty, it pushes tasks from the Microtask Queue or Task Queue into the stack for execution.

Event Loop Execution Flow

1. Start:

- The Call Stack starts empty.
- The JavaScript engine (like V8 in Chrome) loads your script and executes it line by line, stacking functions in the Call Stack as they are called.

2. Execute Synchronous Code:

- All synchronous tasks are executed immediately in the Call Stack.

3. Encounter an Asynchronous Task:

- If an asynchronous operation (like `setTimeout`) is encountered, it's delegated to Web APIs.
- Once the Web API finishes its operation (e.g., waiting for a timer), it sends the associated callback to the Task Queue.

4. Microtask Queue First:

- When the Call Stack is empty, the Event Loop processes all tasks in the Microtask Queue before tasks in the Task Queue.

5. Process the Task Queue:

- If there are no microtasks, the Event Loop picks the first task from the Task Queue and pushes it onto the Call Stack.

JS

```
console.log('Start');
setTimeout(() => {
  console.log('Timeout 1');
}, 0);
Promise.resolve().then(() => {
  console.log('Promise 1');
}).then(() => {
  console.log('Promise 2');
});
setTimeout(() => {
  console.log('Timeout 2');
}, 0);
console.log('End');
```

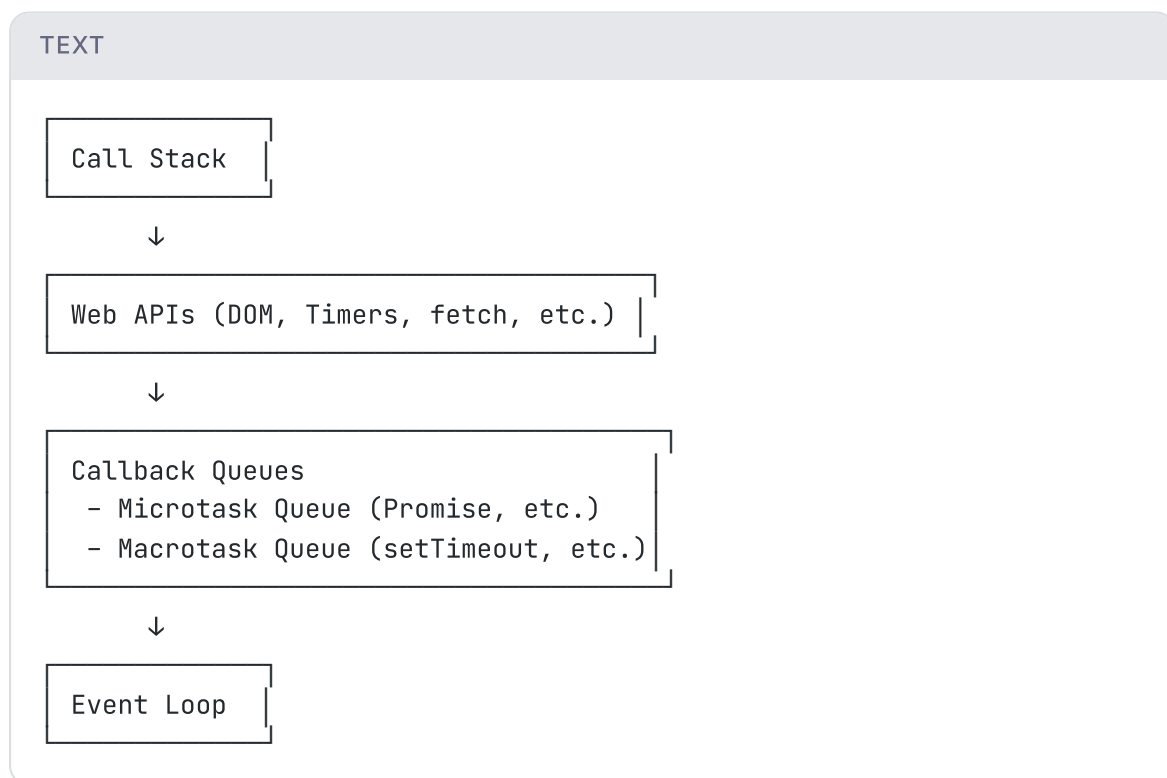
TEXT

```
Start
End
Promise 1
Promise 2
Timeout 1
Timeout 2
```

Step-by-Step Execution

1. **Synchronous Code** — `console.log('Start')` executes immediately.
Output: `Start`
2. **First `setTimeout`** — The callback (Timeout 1) is delegated to the Web API and added to the Task Queue after 0 ms.
3. **Promise Chain** — `Promise.resolve().then()` callback (Promise 1) is added to the Microtask Queue.
The next `.then()` (Promise 2) is added after Promise 1 executes.
4. **Second `setTimeout`** — Its callback (Timeout 2) is added to the Task Queue after 0 ms.
5. **Final Synchronous Code** — `console.log('End')` executes immediately.
Output: `End`
6. **Microtask Queue** — After the Call Stack is empty:
Promise 1 executes → Output: `Promise 1`
Promise 2 executes → Output: `Promise 2`
7. **Task Queue** — After the Microtask Queue is empty:
Timeout 1 executes → Output: `Timeout 1`
Timeout 2 executes → Output: `Timeout 2`

Event Loop Parts



Function of each

- **Call Stack** — Where synchronous JavaScript code is executed.
- **Web APIs** — Handle asynchronous tasks like `setTimeout`, `fetch`, `addEventListener`, etc.
- **Microtask Queue** — For `Promise.then()`, `queueMicrotask()`, etc.
- **Macrotask Queue** — For `setTimeout()`, DOM events, etc.
- **Event Loop** — Monitors and moves tasks from the callback queue to the call stack when it is free.

Other Related Concepts

Modelo de Concurrencia en JavaScript

JavaScript es un lenguaje single-threaded (de un solo hilo), lo que significa que solo puede hacer una cosa a la vez en su flujo principal de ejecución. Sin embargo, a través de la asincronía, JavaScript puede gestionar múltiples tareas de manera eficiente, permitiendo que las operaciones que toman tiempo (como solicitudes de red, temporizadores, etc.) se ejecuten sin bloquear el hilo principal.

V8

V8 is Google's open-source JavaScript engine, used in Chrome and Node.js. It compiles JavaScript to native machine code for fast execution. It includes:

- **Call Stack** — For managing function calls.
- **Heap** — For memory allocation (storing objects and data structures).

Call Stack

A data structure that tracks the active function calls.

- When a function is called, it's added to the top of the stack.
- When the function finishes, it's removed from the stack.
- If the stack is empty, the event loop checks the callback queue for tasks.

Blocking happens if a function on the stack takes a long time to execute, preventing other code from running.

Callback Queue

A queue where callbacks (functions) waiting to be executed are stored.

When asynchronous operations (e.g., `setTimeout`, `fetch`) complete, their callbacks are added to the queue. The event loop dequeues a callback and moves it to the call stack only when the stack is empty.

Web APIs

Browser-provided features (or Node.js equivalents) like `setTimeout`, `fetch`, and DOM events.

These APIs handle tasks asynchronously outside the main thread. When they finish, they place their callbacks into the callback queue (or other queues, like the microtask queue).

Heap

A region of memory used for storing objects and variables. Large data structures are stored here and accessed during code execution.

Asynchronous

Code that doesn't block the call stack.

Examples:

- `setTimeout` schedules a task for later.
- `fetch` performs a network request.

Ensures non-blocking operations by delegating tasks to Web APIs.

Concurrency

JavaScript's ability to manage multiple tasks simultaneously, despite being single-threaded.

While one task is running on the call stack, other tasks (e.g., `fetch`, timers) can progress in Web APIs. When ready, these tasks queue callbacks for execution, ensuring efficient handling of multiple operations.

Blocking

Code that keeps the call stack occupied, preventing other tasks from running.

Example: A `while` loop running indefinitely. It stops the event loop from processing callbacks, causing performance issues.

Functions

Functions in JavaScript are reusable blocks of code that perform a specific task.

Return Value: The value that a function returns is specified using the return keyword. If a function does not have a return statement, it will return undefined by default.

Invocation: To use a function, you must invoke (or call) it by using its name followed by parentheses, which may contain arguments if the function requires them.

Function Declaration

Hoisting: Function declarations in JavaScript are hoisted, meaning they are moved to the top of their scope during compilation. This allows you to call a function before it is defined in the code.

JS

```
function add(a, b) {  
    return a + b;  
}  
add(5, 10); // 15  
greet(); // Hello!  
function greet() {  
    console.log("Hello!");  
}
```

Function Expression

In addition to functions declared with function, you can define functions using function expressions. This is useful when you want to assign a function to a variable.

JS

```
let add = function (a, b) {  
    return a + b;  
};  
console.log(add(2, 3)); // 5
```

No Hoisting: Function expressions are not hoisted. This means the function is not available before the line where it is defined.

JS

```
greet(); // Error: greet is not defined
const greet = function () {
  console.log("Hello!");
};
```

Arrow Functions

Arrow functions offer a more compact syntax and handle the `this` context differently compared to traditional functions.

- If the function has only one parameter, you can omit the parentheses:

JS

```
let print = text => {
  console.log(text);
};
```

- If the function body contains only a single expression, you can omit the curly braces `{}` and the `return` keyword:

JS

```
let add = (num1, num2) => num1 + num2;
```

Hoisting: Arrow functions are created as part of a function expression, so they follow the same hoisting rules as function expressions. This means the function must be defined before calling it.

JS

```
greet(); // Error: greet is not defined
const greet = () => {
  console.log("Hello!");
};
```

Traditional Function:

JS

```
function sum(a, b) {
  return a + b;
}
```

Equivalent Arrow Function:

JS

```
const sum = (a, b) => a + b;
```

Arrow Function with No Parameters:

JS

```
const sayHello = () => console.log("Hello!");
```

Arrow Function with a Single Parameter (No Parentheses Needed):

JS

```
const double = x => x * 2;
```

Immediately Invoked Function Expressions (IIFE)

An IIFE (Immediately Invoked Function Expression) is a function that runs immediately after being defined.

JS

```
(function () {  
    console.log("This is an IIFE");  
})(); // Output: "This is an IIFE"
```

Arrow Functions vs Functions

1. The most important difference is how they handle the value of `this`.
2. Use of arguments.

What is this?

In JavaScript, the `this` keyword refers to an object. The `this` keyword refers to different objects depending on how it is used:

- In an object method, `this` refers to the object.
- Alone, `this` refers to the global object.
- In a function, `this` refers to the global object.
- In a function, in strict mode, `this` is undefined.
- In an event, `this` refers to the element that received the event.
- Methods like `call()`, `apply()`, and `bind()` can refer `this` to any object.

What is Lexical Context?

Lexical context refers to the physical location in the code where a function is defined. It determines which variables and values are available to the function based on its position in the code. This also affects the value of `this` in arrow functions.

A traditional function has its own `this`, which is determined at runtime based on how the function is called.

Arrow Functions: They Do Not Create Their Own `this`

Arrow functions do not define their own `this`. Instead, they inherit the `this` value from the context in which they were declared.

JS

```
const obj = {
  name: "John",
  regularFunction: function () {
    console.log(this.name); // `this` points to the `obj` object
  },
  arrowFunction: () => {
    console.log(this.name); // `this` does not point to `obj`, it inherits from the lexical context
  },
};
obj.regularFunction(); // Output: John
obj.arrowFunction();   // Output: undefined
```

Arrow functions: window or global object.

JS

```
const myFunc = () => {
  console.log(this);
}
myFunc();
```

The arrow function returns the window object (or global in Node.js) because:

- Arrow functions inherit the `this` value from the lexical context where they were defined.
- In the global context, in browsers, the `this` value is the window object.

JS

```
function Timer() {
  this.seconds = 0;
  setInterval(() => {
    this.seconds++; // `this` points to the object created by `new Ti
    console.log(this.seconds);
  }, 1000);
}
const timer = new Timer();
```

Timer is a constructor function. When you use `new Timer()`, a new object is created. Inside the Timer function, the `this` value points to the newly created object. The arrow function inside `setInterval` does not have its own `this`. Instead, it inherits the `this` value from the lexical context in which it was defined, which is the Timer function. Therefore, `this` inside the arrow function refers to the object created by `new Timer()`.

Function Declaration: Strict Mode

JS

```
"use strict";
function example() {
  console.log(this); // undefined
}
example();
```

Global Function Declaration: Global Context

JS

```
function example() {
  console.log(this); // In browsers: window
}
example();
```

Function Declaration: Invoked as a Method

JS

```
const obj = {
  name: "JavaScript",
  greet: function () {
    console.log(this.name);
  },
};
obj.greet(); // "JavaScript"
```

Function Declaration: Constructor Functions

JS

```
function Person(name) {
  this.name = name;
}
const person = new Person("Alice");
console.log(person.name); // "Alice"
```

Function Declaration: DOM Events

JS

```
const button = document.querySelector(".button");
button.addEventListener("click", function () {
  console.log(this); // The button that was clicked
});
```

call, apply, and bind

call() Method

The call method in JavaScript is a function available for all functions because it is part of the Function.prototype. It is used to invoke a function while explicitly specifying the value of this and the arguments to be passed to the function.

Syntax:

JS

```
func.call(thisArg, arg1, arg2, ...)
```

- thisArg: The value you want to assign as this inside the function. It can be any value (an object, an array, null, undefined, etc.). If you pass null or undefined, this is assigned to the global object (window in browsers, global in Node.js).

- `arg1, arg2, ...`: The arguments you want to pass to the function.
- It can be used to invoke (call) a method with an owner object as an argument (parameter).
- The `call` method invokes a function immediately and allows you to explicitly specify the `this` value.

JS

```
function greet(greeting, punctuation) {
  console.log(`${greeting}, ${this.name}${punctuation}`);
}

const person = { name: "Alice" };
greet.call(person, "Hello", "!"); // "Hello, Alice!"

const person = {
  fullName: function () {
    return this.firstName + " " + this.lastName;
  }
}

const person1 = {
  firstName: "John", lastName: "Doe"
}

const person2 = {
  firstName: "Mary", lastName: "Doe"
}

console.log(person.fullName.call(person1)); // John Doe
console.log(person.fullName.call(person2)); // Mary Doe
```

`call` invokes `person.fullName`: The `fullName` function is executed as if it belonged to the object passed as the first argument (in this case, `person1` or `person2`).

apply() Method

Unlike `call`, which requires arguments to be passed individually, `apply` takes the arguments as an array or an array-like object.

Syntax:

JS

```
func.apply(thisArg, [argsArray])
```

- `thisArg`: The value you want to assign to `this` inside the function. If you pass `null` or `undefined`, the global object will be used (window in browsers, global in Node.js).
- `argsArray`: An array or an array-like object containing the arguments for the function.

JS

```
function greet(greeting, age) {
  console.log(`${greeting}, my name is ${this.name} and I'm ${age} years old.`);
}
const person = { name: "Alan" };
greet.apply(person, ["Hello", 26]);
// output: Hello, my name is Alan and I'm 26 years old.
```

Usage with Math:

JS

```
const numbers = [4, 7, 1, 9, 3];
console.log(Math.max.apply(null, numbers)); // 9
console.log(Math.min.apply(null, numbers)); // 1
```

null is used as this because Math.max does not require a context.

JS

```
function sum(a, b, c) {
  return a + b + c;
}
console.log(sum.call(null, 1, 2, 3)); // 6 (with call)
console.log(sum.apply(null, [1, 2, 3])); // 6 (with apply)
```

Inheritance Simulation:

JS

```
function Person(name, age) {
  this.name = name;
  this.age = age;
}
function Employee(name, age, position) {
  Person.apply(this, [name, age]);
  this.position = position;
}
const employee = new Employee("John", 30, "Developer");
console.log(employee); // Employee { name: 'John', age: 30, position: 'Developer' }
```

apply allows Employee to reuse the constructor of Person.

bind() Method

The bind() method creates a new function with a specific this value, and optionally, pre-set arguments.

Syntax:

JS

```
const boundFunction = func.bind(thisArg[, arg1[, arg2[, ...]]]);
```

- func: The original function you want to bind (bind does not modify the original).
- thisArg: The value you want to assign to this in the new function.
- arg1, arg2, ...: Optional. Predefined arguments that will be passed when the new function is invoked.

JS

```
const person = {
  firstName: "John",
  lastName: "Doe",
  fullName: function () {
    return this.firstName + " " + this.lastName;
  }
}
const member = {
  firstName: "Hege",
  lastName: "Nilsen",
}
let fullName = person.fullName.bind(member);
console.log(fullName()); // Hege Nilsen
```

Lost Methods: When you extract a method from an object, the value of this can be lost. Using bind ensures that the method works correctly.

JS

```
const counter = {
  value: 0,
  increment() {
    this.value++;
    console.log(this.value);
  }
};
const increment = counter.increment.bind(counter);
increment(); // 1
increment(); // 2
```

JS

```
function introduce(name, age, city) {
    return `${this.title} ${name} is ${age} years old and lives in ${city}
}
const person = { title: "Mr." };

const introduceWithTitle = introduce.bind(person, "Alan", 25);
console.log(introduceWithTitle("Mexico City"));
// Output: "Mr. Alan is 25 years old and lives in Mexico City."
```

JS

```
function sum(a, b, c) {
    return a + b + c;
}
const sumWith5 = sum.bind(null, 5);
console.log(sumWith5(3, 7)); // Output: 15
```

Use of arguments

- arguments is an array-like object that is automatically available inside any traditional function (but not in arrow functions).
- It contains all the arguments passed to the function, regardless of how many are explicitly defined in the function declaration.
- Although arguments is not a true array (it's an object), you can convert it into an array if needed.

JS

```
function sum() {
    console.log(arguments);
    // [Arguments] { '0': 1, '1': 2, '2': 3 }
    let total = 0;
    for (let i = 0; i < arguments.length; i++) {
        total += arguments[i];
    }
    return total;
}
console.log(sum(1, 2, 3)); // 6
```

Limitations of arguments

Not a real array: arguments is an object, not an array. It doesn't have array methods like .map(), .filter(), etc. However, you can convert it into an array.

JS

```
function sum() {  
  const args = Array.from(arguments); // Convert to array  
  return args.reduce((total, num) => total + num, 0);  
}  
console.log(sum(1, 2, 3)); // 6
```

arguments in Arrow Functions

arguments is not available in arrow functions. If you try to use it, you will get an error because arrow functions do not have their own arguments object.

JS

```
const sum = () => {  
  console.log(arguments); // Error: arguments is not defined  
};  
sum(1, 2, 3);
```

Modern Alternative: Rest Parameters

Instead of using arguments, you can use rest parameters (...) in modern functions. These allow you to collect all arguments into a real array.

JS

```
const sum = (...args) => {  
  return args.reduce((total, num) => total + num, 0);  
};  
console.log(sum(1, 2, 3)); // 6
```

JS

```
function sum(...args) {  
  return args.reduce((total, num) => total + num, 0);  
}  
console.log(sum(1, 2, 3)); // 6
```

Parameters Omission

You can use `_` as a placeholder name for unused parameters. This is a convention that signals the parameter is intentionally not used.

JS

```
const myFunction = (_, secondParam) => {  
  console.log(secondParam);  
};  
  
myFunction("No lo uso", "Este sí se imprime"); // Output: "Este sí se imp  
  
const numbers = [10, 20, 30];  
const indexedNumbers = numbers.map( (_, index) => index);  
  
console.log(indexedNumbers); // Output: [0, 1, 2]
```

JavaScript Fundamentals

Primitive types

- String Represents a text string. single quotes ('), double quotes ("), o backticks (`).

JS

```
let name = "John";
let name2 = 'John';
let greeting = `Hello, ${name}!`;
```

- Number Represents numbers, either integers or decimals.

JS

```
let age = 25;
let price = 19.99;
```

- BigInt Allows working with integers larger than those that the Number type can represent. A n is added at the end of an integer to convert it into a BigInt.

JS

```
let bigNumber = 123456789012345678901234567890n;
```

- Boolean Represents a truth value that can be true or false.

JS

```
let isAdult = true;
let isMinor = false;
```

- Undefined Represents the absence of an assigned value. A declared but uninitialized variable will have the value undefined.

JS

```
let x;
console.log(x); // undefined
```

- Symbol Represents a unique value and is used as a unique identifier for object properties. Symbols are unique, even if two Symbols have the same description.

JS

```
let id = Symbol("id");
let anotherId = Symbol("id");
console.log(id === anotherId); // false
```

- null Represents the intentional absence of a value. Unlike undefined, null is explicitly assigned to a variable to indicate that it has no value.

JS

```
let person = null;
```

Objects

In JavaScript, objects can be seen as a collection of properties. Property values can be values of any type. It is composed of key-value pairs.

JS

```
const person = {
  name: 'John Doe',
  age: 30,
  address: '123 Main Street'
};
```

Arrays

The Array object, as with arrays in other programming languages, enables storing a collection of multiple items under a single variable name, and has members for performing common array operations.

JS

```
const arr = [30, 40, 10, true, "Hello"];
```

Variables

var

var is the oldest way to declare variables in JavaScript. It has function scope or global scope, depending on where it is declared. Hoisting: Variables declared with var are hoisted to the top of their context (function or global), meaning they can be accessed

before their declaration, although their value will be undefined. Scope (alcance): var has function scope, meaning if declared inside a function, it won't be accessible outside that function. If declared outside a function, it becomes accessible throughout the global context.

JS

```
function exampleVar() {  
  console.log(x); // undefined (hoisting)  
  var x = 10;  
  console.log(x); // 10  
}  
exampleVar();
```

let

let allows you to declare variables with block scope, meaning the variable is only accessible within the {} block where it was declared. No hoisting: Although let declarations are hoisted, they are not initialized. If you try to access a variable declared with let before its initialization, you'll get a ReferenceError.

JS

```
console.log(y); // ReferenceError: Cannot access 'y' before initialization  
let y = 20;  
console.log(y); // 20
```

var vs let

var has global scope if declared outside a function. let has block scope, meaning it's only accessible within the {} block where it was declared.

JS

```
// Using var - Global scope
const arr = [1, 2, 3];
function sum(arr) {
    for (var i = 0; i < arr.length; i++) {
        // some code
    }
    // variable i is available here.
    console.log("value of i with var:", i); // 3
}
sum(arr);
// Using let - Block scope
function sum2(arr) {
    for (let i = 0; i < arr.length; i++) {
        // some code
    }
    // variable i is not available here.
    console.log("value of i with let:", i); // undefined
}
sum2(arr);
```

const

const is immutable (regarding the reference): The reference of the variable cannot be changed, meaning it cannot be reassigned to another value or object. However, if the variable is an object or array, its internal properties or elements can be modified. Scope: const has block scope.

JS

```
function exampleConst() {
    const z = 30;
    console.log(z); // 30

    // z = 40; // Error: Assignment to constant variable

    const obj = { name: "Juan" };
    obj.name = "Pedro"; // This is allowed
    console.log(obj.name); // Pedro
}
exampleConst();
```

Hoisting

Hoisting is a JavaScript behavior where variable and function declarations are moved (or "hoisted") to the top of their containing scope during compilation, before the code is

executed. This means you can use a function or variable before declaring it in the code. Functions and variables are assigned in memory at the moment of compilation. Basically, this allows us to use functions before they were declared.

JS

```
greet(); // Works even though it's called before declaration
function greet() {
  console.log("Hello!");
}
console.log(a); // undefined (declaration is hoisted, but not initialized)
var a = 10;
console.log(a); // 10
```

Temporal Dead Zone

The temporal dead zone (TDZ) is a specific period in the execution of JavaScript code where variables declared with `let` and `const` exist but cannot be accessed or assigned any value. During this phase, accessing or using the variable will result in a `ReferenceError`.

- The TDZ starts from the beginning of the block until the variable is declared.
- Variables declared with `let` and `const` are hoisted but not initialized.
- Accessing the variable in the TDZ results in a `ReferenceError`.
- `var` declarations do not have a TDZ and are initialized as `undefined`.

JS

```
console.log(b); // ReferenceError: Cannot access 'b' before initialization
let b = 20;
```

Scope

Scope determines the accessibility (visibility) of variables. JavaScript variables have 3 types of scope:

- Block scope
- Function scope
- Global scope

Block Scope

Variables declared inside a `{ }` block cannot be accessed from outside the block.

JS

```
{  
    let x = 2;  
}  
// x can NOT be used here
```

Local Scope

Variables declared within a JavaScript function, are LOCAL to the function:

JS

```
// code here can NOT use carName  
function myFunction() {  
    let carName = "Volvo";  
    // code here CAN use carName  
}  
// code here can NOT use carName
```

Global JavaScript Variables

A variable declared outside a function, becomes GLOBAL.

JS

```
let carName = "Volvo";  
// code here can use carName  
function myFunction() {  
    // code here can also use carName  
}
```

JavaScript Modules

Los módulos permiten organizar y dividir el código en partes reutilizables y mantenibles. Un módulo es un archivo que puede contener funciones, variables, objetos o clases, y se puede importar y exportar en otros archivos.

ES Modules

Nativos de JavaScript, funcionan en navegadores modernos y Node.js.

JS

```
<script src="./js/17-modules.js" type="module"></script>
```

exports.js

JS

```
export const heroes = [
  {
    id: 1,
    name: 'Batman',
    owner: 'DC'
  },
  {
    id: 2,
    name: 'Spiderman',
    owner: 'Marvel'
  },
  {
    id: 3,
    name: 'Superman',
    owner: 'DC'
  },
  {
    id: 4,
    name: 'Flash',
    owner: 'DC'
  },
  {
    id: 5,
    name: 'Wolverine',
    owner: 'Marvel'
  },
];

export function sumar(a, b) {
  console.log(a + b);
}

export default function Multiplicar(a, b) {
  console.log(a * b);
}

/*
export {
  sumar,
  heroes
}
*/
```

app.js

JS

```
import Multiplicar, { heroes, sumar } from "../js/17-exports.js";

console.log(heroes); // (5) [{...}, {...}, {...}, {...}, {...}]
console.log(heroes[0].name); // Batman
sumar(2, 2); // 4
Multiplicar(6, 6); // 36
```

CommonJS

Sistema de módulos utilizado principalmente en Node.js.

exports.js

JS

```
const heroes = [
  { id: 1, name: 'Batman', owner: 'DC' },
  { id: 2, name: 'Spiderman', owner: 'Marvel' },
  { id: 3, name: 'Superman', owner: 'DC' },
  { id: 4, name: 'Flash', owner: 'DC' },
  { id: 5, name: 'Wolverine', owner: 'Marvel' },
];

function sumar(a, b) {
  console.log(a + b);
}

function Multiplicar(a, b) {
  console.log(a * b);
}

module.exports = {
  heroes,
  sumar,
  Multiplicar,
};
```

app.js

JS

```
const { heroes, sumar, Multiplicar } = require('./exports');

console.log(heroes);
console.log(heroes[0].name); // Batman
sumar(2, 2); // 4
Multiplicar(6, 6); // 36
```

Diferencia clave

ES Modules	CommonJS
<code>export</code> / <code>import</code>	<code>module.exports</code> / <code>require()</code>
Estático (las importaciones se analizan en tiempo de compilación)	Dinámico (se evalúa en tiempo de ejecución)
<code>import</code> es asíncrono	<code>require()</code> es síncrono
Usado en navegadores y Node.js moderno	Usado principalmente en Node.js

Objects & Copying

Object Literals

An object is a collection of data and functionalities represented as key-value pairs.

JS

```
let person = {  
  name: "Juan",  
  age: 30,  
  isStudent: true,  
  greet: function () {  
    console.log("Hello, my name is " + this.name);  
  }  
};
```

Accessing Properties

Dot Notation:

JS

```
console.log(person.name); // "Juan"  
person.age = 31;  
console.log(person.age); // 31
```

Bracket Notation: Useful when the property name is dynamic or doesn't follow variable naming conventions.

JS

```
console.log(person["name"]); // "Juan"  
person["age"] = 32;  
console.log(person["age"]); // 32
```

Object Methods

JS

```
person.greet(); // "Hello, my name is Juan"
```

Updating a Value

JS

```
person["age"] = 32;  
person.age = 32;
```

Adding New Properties

If the property does not exist, it will be added to the object.

JS

```
person.lastName = "Corona";
```

Deleting a Property

JS

```
delete person.lastName;
```

Freezing an Object

JS

```
Object.freeze(person); // After freezing, you cannot add, remove, or modify
```

Sealing an Object

It prevents adding or deleting properties, but allows modifying existing ones.

JS

```
Object.seal(person);
```

Merging Two or More Objects

Using Spread Operator

JS

```
const product = {
  name: 'Tablet',
  price: 50
};
const customer = {
  name: 'John',
  premium: true
};
const newObject = {
  product: { ...product },
  customer: { ...customer }
};
console.log(newObject);
// Output:
// {
//   product: { name: 'Tablet', price: 50 },
//   customer: { name: 'John', premium: true }
// }
```

JS

```
const newObject = {
  ...product,
  ...customer
}
console.log(newObject);
// output: { name: 'John', price: 50, premium: true }
```

Using Object.assign()

JS

```
const product = {
  name: "20-inch Monitor",
  price: 300,
  available: true,
};
const dimensions = {
  weight: '1kg',
  size: '1m',
};
const result = Object.assign(product, dimensions);
console.log(result);
// output:
// {
//   name: '20-inch Monitor',
//   price: 300,
//   available: true,
//   weight: '1kg',
//   size: '1m'
// }
```

Copying by Reference vs Shallow Copy vs Deep Copy

Copying by Reference

When you assign an object or array to a new variable, you're copying the reference, not the actual data.

JS

```
const originalArray = [1, 2, 3];
const copiedArray = originalArray; // Copying the reference
copiedArray.push(4);
console.log(originalArray); // [1, 2, 3, 4] - The original is modified
console.log(copiedArray); // [1, 2, 3, 4] - Both point to the same underlying memory
```

Shallow Copy

A shallow copy occurs when you copy the reference of an object to a new variable. In this process, only the top-level properties are copied, while nested objects or arrays still reference the original memory location. This means that if you change the nested properties in one object, those changes will reflect in the other because they share the same memory reference.

Spread Operator (...)

JS

```
const person = {
  "name": 'John',
  "age": 20,
};
const personcopy = { ...person };
console.log("person", person);
console.log("personcopy", personcopy);
personcopy.name = "Alan";
console.log("person", person);
console.log("personcopy", personcopy);
// person { name: 'John', age: 20 }
// personcopy { name: 'John', age: 20 }
// person { name: 'John', age: 20 }
// personcopy { name: 'Alan', age: 20 }
```

Disadvantage: Just like `Object.assign()`, this doesn't copy nested objects deeply.

Object.assign()

JS

```
const obj = {
  name: 'Version 1',
  additionalInfo: {
    version: 1
  }
};
const shallowCopy1 = { ...obj };
const shallowCopy2 = Object.assign({}, obj);
shallowCopy1.name = 'Version 2';
shallowCopy1.additionalInfo.version = 2;
shallowCopy2.name = 'Version 3';
shallowCopy2.additionalInfo.version = 3;
console.log(obj); // { name: 'Version 1', additionalInfo: { version: 3 } }
console.log(shallowCopy1); // { name: 'Version 2', additionalInfo: { vers
console.log(shallowCopy2); // { name: 'Version 3', additionalInfo: { vers
```

Disadvantage: It doesn't copy deep nested objects; it only copies references to the internal objects.

Deep Copy

A deep copy creates a completely independent copy of the object, including all nested objects or arrays. This ensures that changes made to one object do not affect the other.

JSON.parse(JSON.stringify())

JS

```
const obj = { name: 'Version 1', additionalInfo: { version: 1 } };
const deepCopy = JSON.parse(JSON.stringify(obj));
deepCopy.name = 'Version 2';
deepCopy.additionalInfo.version = 2;
console.log(obj); // { name: 'Version 1', additionalInfo: { version: 1 } }
console.log(deepCopy); // { name: 'Version 2', additionalInfo: { version: 2 } }
```

Advantage: It works well for simple structures. Disadvantage: It doesn't copy functions, undefined, Symbol, BigInt, or circular references.

structuredClone()

It can perform deep copies of as many levels as needed; it creates a completely new copy of the original object without shared references or properties.

JS

```
const obj = {
  name: "Alan",
  details: {
    age: 25,
    skills: ["JavaScript", "TypeScript"],
  },
};
const copy = structuredClone(obj);
copy.details.age = 30;
console.log(obj.details.age); // 25 (The original doesn't change)
console.log(copy.details.age); // 30 (The clone is independent)
```

You can clone objects that have circular references, which is something not possible with the JSON approach.

JS

```
const circularObj = { name: "Circular" };
circularObj.self = circularObj; // Circular reference
const clone = structuredClone(circularObj);
console.log(clone.self === clone); // true (Deep copy preserves the reference)
```

However, it cannot clone functions or methods.

JS

```
const objWithFunc = {
  greet() {
    return "Hello!";
  },
};
const clonedObj = structuredClone(objWithFunc);
console.log(clonedObj.greet); // undefined (The function is lost)
```

Summary

Method	Copy type	Support nested	Support Date, Map, Set	Support functions
Object.assign()	Shallow (copy references)	No	Yes	Yes
Spread operator	Shallow (copy references)	No	Yes	Yes
JSON.parse()	Deep (copy values)	No	No	No
structuredClone()	Deep (copy values)	Yes	Yes	No

Destructuring

The destructuring assignment syntax unpacks object properties into variables.

Object Destructuring

JS

```
const person = {
  name: "John",
  lastName: "Doe",
  age: 50
};
const product = {
  name: "tablet",
  price: 50,
};

let { name, lastName } = person;
```

Avoid variable name collisions:

JS

```
let { name: productName } = product;
```

Default Values

JS

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 50  
};  
let { firstName, lastName, country = "US" } = person;
```

Destructuring Nested Objects

JS

```
const client = {  
  name: "Alan",  
  premium: true,  
  address: {  
    street: "Av.México"  
  }  
};  
const { name: clientName, address: { street } } = client;  
console.log(street); // "Av.México"
```

Destructuring and Renaming

JS

```
const user = {  
  name: "Juan",  
  address: {  
    city: "Madrid",  
    postalCode: "28001"  
  }  
};  
const { address: { city: userCity } } = user;  
console.log(userCity); // "Madrid"
```

The Rest Operator

It allows us to collect and gather "the rest" of the properties of an object into a new object.

JS

```
const person = {
  name: 'John',
  age: 30,
  profession: 'Developer',
  country: 'USA'
};
const { name, age, ...rest } = person;
console.log(name); // John
console.log(age); // 30
console.log(rest); // { profession: 'Developer', country: 'USA' }
```

Array Destructuring

We can pick up array variables into our own variables.

JS

```
const fruits = ["Bananas", "Oranges", "Apples", "Mangos"];
let [fruit1, fruit2] = fruits;
console.log(fruit1, fruit2); // Bananas Oranges
```

Skipping Array Values

We can skip array values using two or more commas.

JS

```
const fruits = ["Bananas", "Oranges", "Apples", "Mangos"];
let [fruit1,,,fruit2] = fruits;
console.log(fruit1, fruit2); // Bananas Mangos
```

Array Position Values

We can pick up values from specific index locations of an array.

JS

```
const fruits = ["Bananas", "Oranges", "Apples", "Mangos"];
let {[3]: fruit1, [1]: fruit2} = fruits;
console.log(fruit2, fruit1); // Oranges Mangos
```

The Rest Operator in Arrays

You can end a destructuring syntax with a rest property. This syntax will store all remaining values into a new array.

JS

```
const numbers = [10, 20, 30, 40, 50, 60, 70];  
const [a, b, ...rest] = numbers;  
console.log(a, b); // 10 20  
console.log(rest); // [ 30, 40, 50, 60, 70 ]
```

Operators

Arithmetic Operators

These operators are used to perform mathematical operations on numbers.

JS

```
let a = 10 + 5; // 15
let b = 10 - 5; // 5
let c = 10 * 5; // 50
let d = 10 / 2; // 5
let e = 10 % 3; // 1 (remainder / modulo)
let f = 2 ** 3; // 8 (2 raised to the power of 3)
let g = 5;
g++; // g is now 6
let h = 5;
h--; // h is now 4
```

Assignment Operators

JS

```
let x = 10; // Assigns the value 10 to x
x += 5; // Equivalent to x = x + 5; x is now 15
x -= 3; // Equivalent to x = x - 3; x is now 12
x *= 2; // Equivalent to x = x * 2; x is now 24
x /= 4; // Equivalent to x = x / 4; x is now 6
x %= 5; // Equivalent to x = x % 5; x is now 1
```

Comparison Operators

== Equality | Abstract Equality (compares only the value)

JS

```
let isEqual = (5 == "5"); // true (compares the value without considering
```

=== Strict Equality (compares both value and type)

JS

```
let isStrictEqual = (5 === "5"); // false (compares both the value and th
```

!= Inequality (compares only the value)

JS

```
let isDifferent = (5 !== "5"); // false (compares the value without consid
```

!== Strict Inequality (compares both value and type)

JS

```
let isStrictDifferent = (5 === "5"); // true (compares both the value and
```

JS

```
let greaterThan = (10 > 5); // true
let lessThan = (10 < 5); // false
let greaterOrEqual = (10 ≥ 10); // true
let lessOrEqual = (10 ≤ 5); // false
```

Logical Operators

&& Logical AND

JS

```
let result = (true && false); // false (both operands must be true)
```

|| Logical OR

JS

```
let result = (true || false); // true (at least one of the operands must
```

! Logical NOT

JS

```
let result = !true; // false (negates the boolean value)
```

Type Operators

typeof: Devuelve el tipo de una variable o expresión.

JS

```
let type = typeof 42; // "number"
let typeStr = typeof "Hello"; // "string"
```

instanceof: Verifica si un objeto es una instancia de un constructor específico.

JS

```
let date = new Date();
let isInstance = date instanceof Date; // true
```

Conditional (Ternary) Operator

JS

```
condition ? valueIfTrue : valueIfFalse
let age = 18;
let isAdult = (age ≥ 18) ? "Yes" : "No"; // "Yes"
```

Optional Chaining (?.)

Permite acceder de manera segura a propiedades de objetos anidados o llamar a funciones que podrían no existir, sin tener que verificar manualmente si cada propiedad intermedia está definida.

JS

```
const adventurer = {
  name: 'Alice',
  cat: {
    name: 'Dinah',
  },
};
const dogName = adventurer.dog?.name;
console.log(dogName);
// Expected output: undefined
console.log(adventurer.someNonExistentMethod?.());
// Expected output: undefined
```

Nullish Coalescing Operator (??)

Es un operador lógico que devuelve el valor de su operando derecho cuando su operando izquierdo es null o undefined. Si el operando izquierdo no es null ni undefined, entonces devuelve el valor de ese operando izquierdo.

JS

```
let userPreference = null;
let defaultPreference = 'dark mode';
let currentPreference = userPreference ?? defaultPreference;
console.log(currentPreference); // 'dark mode'
let client = "John";
console.log(client ?? "Anonymous"); // John (user no es null ni undefined)
```

Short-circuit (&&) Evaluation

Evalúa los operandos de izquierda a derecha y devuelve el primer operando que sea falsy (es decir, un valor que se evalúa como false en un contexto booleano). Si ambos operandos son truthy, devuelve el valor del último operando.

JS

```
let result = 0 && 'Hello';
console.log(result); // 0
let result2 = 'Hi' && 'Hello'; // hi y hello son truthy, devuelve hello
console.log(result2); // 'Hello'
```

Truthy & Falsy


```
// Falsy
Boolean(false);    // false
Boolean(undefined); // false
Boolean(null);     // false
Boolean("");       // false
Boolean('');       // false
Boolean(``);       // false
Boolean(NaN);      // false
Boolean(0);        // false
Boolean(-0);       // false
Boolean(0n);       // false

// Truthy
Boolean("false");   // true
Boolean(true);      // true
Boolean([]);        // true
Boolean({});        // true
Boolean(new Date()); // true
Boolean(" ");       // true (espacio)
Boolean(43);        // true
Boolean(-3.14);     // true
Boolean(3.14);      // true
Boolean(Infinity);  // true
Boolean(-Infinity); // true
```

JavaScript Promises

A Promise in JavaScript is an object representing the eventual completion (or failure) of an asynchronous operation and its resulting value.

A promise may have one of three states:

- **Pending**
- **Fulfilled**
- **Rejected**

A promise starts in a pending state. If the operation is successful, it ends in a fulfilled state. If an error occurs, it ends in a rejected state.

```
const heroes = [
  {
    id: 1,
    name: 'Batman',
    owner: 'DC'
  },
  {
    id: 2,
    name: 'Spiderman',
    owner: 'Marvel'
  },
  {
    id: 3,
    name: 'Superman',
    owner: 'DC'
  },
  {
    id: 4,
    name: 'Flash',
    owner: 'DC'
  },
  {
    id: 5,
    name: 'Wolverine',
    owner: 'Marvel'
  },
];

const getHeroById = (id) => {
  return heroes.find(hero => hero.id === id);
};

const getHeroByIdAsync = (id) => {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      const p1 = getHeroById(id);
      if (p1) {
        resolve(p1);
      } else {
        reject('No existe el heroe con ese ID');
      }
    }, 2000);
  });
};

getHeroByIdAsync(10)
```

```
.then(console.log)
.catch(console.warn);
```

JS

```
function obtenerDatosUsuario(idUsuario) {
  return new Promise((resolve, reject) => {
    fetch(`https://jsonplaceholder.typicode.com/users/${idUsuario}`)
      .then((response) => {
        if (!response.ok) {
          reject('Error al obtener los datos del usuario');
        }
        return response.json();
      })
      .then((datos) => {
        resolve(datos);
      })
      .catch((error) => {
        reject(error.message);
      });
  });
}

function mostrarDatosUsuario(datos) {
  console.log(`Nombre del usuario: ${datos.name}`);
  console.log(`Username: ${datos.username}`);
  console.log(`Email: ${datos.email}`);
  console.log(`Ciudad: ${datos.address.city}`);
}

obtenerDatosUsuario(1)
  .then((datos) => {
    mostrarDatosUsuario(datos);
  })
  .catch((error) => {
    console.error('Error:', error);
  })
  .finally(() => {
    console.log('Operación completada.');
```

Promise.all

Es un método que te permite ejecutar múltiples promesas en paralelo y esperar a que todas se resuelvan o a que alguna falle.

Sintaxis

JS

```
Promise.all([promise1, promise2, promise3, ...])
  .then((results) => {
    // results es un arreglo con los resultados de cada promesa resuelta
  })
  .catch((error) => {
    // Si alguna promesa falla, se ejecuta este bloque
  });
```

Ejemplo

JS

```
const promise1 = new Promise((resolve, reject) => {
  setTimeout(() => resolve('Tarea 1 completada'), 1000);
});
const promise2 = new Promise((resolve, reject) => {
  setTimeout(() => resolve('Tarea 2 completada'), 2000);
});
const promise3 = new Promise((resolve, reject) => {
  setTimeout(() => resolve('Tarea 3 completada'), 3000);
});

Promise.all([promise1, promise2, promise3])
  .then((results) => {
    console.log('Todas las tareas se completaron:', results);
  })
  .catch((error) => {
    console.log('Algunas tareas fallaron:', error);
  });
```

Output:

TEXT

Todas las tareas se completaron: ['Tarea 1 completada', 'Tarea 2 completada', 'Tarea 3 completada']

NOTA: Si alguna de las promesas es rechazada, `Promise.all()` detendrá su ejecución y pasará directamente al bloque `.catch()` con el error de la primera promesa que falló.

Promise.all con Fetch API

JS

```
function getPosts(urls) {
  const fetchPromises = urls.map(url =>
    fetch(url)
      .then(response => {
        if (!response.ok) {
          throw new Error('Error en la solicitud: ' + response.
        }
        return response.json();
      })
      .catch(error => {
        console.error(`Error con la URL ${url}:`, error);
        return null;
      })
  );

  return Promise.all(fetchPromises)
    .then(responses => {
      return responses.filter(response => response !== null);
    })
    .catch(error => {
      console.error('Hubo un problema con las solicitudes:', error);
      throw error;
    });
}

const urls = [
  "https://jsonplaceholder.typicode.com/posts/1",
  "https://jsonplaceholder.typicode.com/posts/2",
  "https://jsonplaceholder.typicode.com/posts/3",
];

getPosts(urls).then((contents) => {
  console.log("posts: ", contents);
});
```

Promise.all con async/await

JS

```
async function fetchPosts(urls) {
  try {
    const responses = await Promise.all(urls.map(url =>
      fetch(url).then(response => response.json())
    ));
    console.log("Posts obtenidos con async/await:", responses);
  } catch (error) {
    console.error("Hubo un error:", error);
  }
}

const urls = [
  "https://jsonplaceholder.typicode.com/posts/1",
  "https://jsonplaceholder.typicode.com/posts/2",
  "https://jsonplaceholder.typicode.com/posts/3",
];
fetchPosts(urls);
```

Prototypes & Classes

Prototypes

A prototype is an internal object from which other objects inherit properties and methods. In JavaScript, every object has an internal property called `[[Prototype]]` (often accessible via **proto**).

- `__proto__` : A property of objects that points to their prototype, deprecated in favor of `Object.getPrototypeOf()`

When you try to access a property or method on an object and it doesn't exist in that object, JavaScript looks for it in the object's prototype. If it's not found there, it continues searching up the chain of prototypes until it reaches the base object, `Object.prototype`.

`Object.prototype` is a built-in object from which all other objects inherit. It acts as the "parent" of all objects in JavaScript. The prototype of `Object.prototype` is null, which indicates the end of the prototype chain.

Object.prototype methods

- `toString()` : Converts an object to a string representation.
- `valueOf()` : Returns the primitive value of an object.
- `hasOwnProperty(prop)` : Checks if an object directly owns a given property.
- `isPrototypeOf(obj)` : Checks if the current object is in the prototype chain of another object.
- `propertyIsEnumerable(prop)` : Checks if a property is enumerable on the object.

How Prototypes Work

- **Prototype Chain:** When you try to access a property on an object, JavaScript first looks for it on the object itself. If not found, it looks up the prototype chain until it finds the property or reaches the end of the chain (where the prototype is null).
- **Constructor Functions:** When you create an object using a constructor function (with `new`), the new object's prototype is set to the constructor's prototype property.

Accessing an Object's Prototype

JS

```
const person = {
  name: 'John',
  greet: function () {
    console.log('Hello, ' + this.name);
  }
};
console.log(person.__proto__);
// person.__proto__ points to Object.prototype, the default prototype for
```

Prototype with Constructor Functions

Constructor functions have a prototype (Constructor.prototype). This prototype is used to share methods among all instances created by that constructor.

JS

```
function Person(name, age) {
  this.name = name;
  this.age = age;
}

Person.prototype.greet = function () {
  console.log(`Hello, my name is ${this.name} and I'm ${this.age} years old`);
};

const john = new Person('John', 30);
john.greet(); // Hello, my name is John and I'm 30 years old.

const mary = new Person('Mary', 25);
mary.greet(); // Hello, my name is Mary and I'm 25 years old.
```

The john object has a prototype chain:

- `john.__proto__` points to `Person.prototype`
- `Person.prototype.__proto__` points to `Object.prototype`
- `Object.prototype.__proto__` points to `null`

The Prototype of Object

All JavaScript objects inherit from `Object.prototype` (unless created explicitly without a prototype using `Object.create(null)`).

JS

```
const obj = { name: 'Object' };  
console.log(obj.__proto__ === Object.prototype); // true
```

Modifying an Object's Prototype

You can modify a constructor's prototype to add methods to all instances.

JS

```
function Animal(name) {  
  this.name = name;  
}  
  
Animal.prototype.speak = function () {  
  console.log(this.name + ' makes a sound.');};  
  
const dog = new Animal('Dog');  
dog.speak(); // Dog makes a sound.  
  
Animal.prototype.speak = function () {  
  console.log(this.name + ' barks.');};  
  
const cat = new Animal('Cat');  
cat.speak(); // Cat barks.
```

`__proto__` VS `prototype`

- `__proto__`: A property of objects used to access the prototype of an object.
- `prototype`: A property of constructor functions that defines what is shared among all instances.

JS

```
function Car(make, model) {  
  this.make = make;  
  this.model = model;  
}  
  
const myCar = new Car('Toyota', 'Corolla');  
console.log(myCar.__proto__ === Car.prototype); // true
```

Note: `__proto__` is a legacy feature and should not be used in production

code.

Prototype Inheritance

JS

```
function Animal(name) {
  this.name = name;
}

Animal.prototype.speak = function () {
  console.log(`${this.name} makes a sound.`);
};

function Dog(name, breed) {
  Animal.call(this, name);
  this.breed = breed;
}

Dog.prototype = Object.create(Animal.prototype);
Dog.prototype.constructor = Dog;

Dog.prototype.bark = function () {
  console.log(`${this.name}, the ${this.breed}, barks!`);
};

const myDog = new Dog('Buddy', 'Golden Retriever');
myDog.speak(); // Buddy makes a sound.
myDog.bark(); // Buddy, the Golden Retriever, barks!

console.log(myDog instanceof Dog); // true
console.log(myDog instanceof Animal); // true
```

JS

```
function Parent(name) {
  this.name = name;
}

Parent.prototype.greet = function () {
  console.log(`Hello, I am ${this.name}.`);
};

function Child(name, age) {
  Parent.call(this, name);
  this.age = age;
}

Child.prototype = Object.create(Parent.prototype);
Child.prototype.constructor = Child;

Child.prototype.sayAge = function () {
  console.log(`I am ${this.age} years old.`);
};

const child = new Child("Alice", 10);
child.greet(); // "Hello, I am Alice."
child.sayAge(); // "I am 10 years old."
```

Prototype Chain



Prototype chain diagram 1



Prototype chain diagram 2

When you access a property or method on an object:

1. JavaScript first looks for it on the object itself.
2. If not found, it looks at the object's `[[Prototype]]`.
3. This continues up the chain until it reaches `Object.prototype`.
4. If it's not there, it returns undefined.

JS

```
const obj = {};
console.log(obj.toString()); // [object Object]
console.log(obj.__proto__); // Object.prototype
console.log(Object.prototype.__proto__); // null
```

Prototype Methods

`Object.create(proto)`

Creates a new object with the specified prototype.

JS

```
const animal = {
  type: 'Animal',
  makeSound() {
    console.log('Generic animal sound');
  }
};

const dog = Object.create(animal);
dog.bark = function () {
  console.log('Woof!');
};

console.log(dog.type); // "Animal" (inherited)
dog.makeSound(); // "Generic animal sound" (inherited)
dog.bark(); // "Woof!"
```

Object.getPrototypeOf(obj)

Retrieves the prototype of a given object.

JS

```
const animal = {
  type: 'Animal',
};

const dog = Object.create(animal);
console.log(Object.getPrototypeOf(dog)); // {type: 'Animal'}
console.log(dog.__proto__); // {type: 'Animal'}
```

Object.setPrototypeOf(obj, proto)

Sets the prototype of a specified object. Warning: this is a slow operation.

JS

```
const originalPrototype = { greeting: "Hello" };
const myObject = Object.create(originalPrototype);
const newPrototype = { farewell: "Goodbye" };

Object.setPrototypeOf(myObject, newPrototype);
console.log(myObject.greeting); // undefined
console.log(myObject.farewell); // "Goodbye"
```

Object.isPrototypeOf(obj)

Checks if an object exists in another object's prototype chain.

JS

```
const animal = {
  type: 'Animal',
};

const dog = Object.create(animal);
console.log(animal.isPrototypeOf(dog)); // true
console.log(Object.prototype.isPrototypeOf(dog)); // true
console.log(dog.isPrototypeOf(animal)); // false
```

How to Remove the Prototype of an Object

You can use `Object.setPrototypeOf()` to set an object's prototype to null. This disconnects the object from the prototype chain.

Effects of Removing the Prototype:

- Methods like `toString()`, `hasOwnProperty()`, or `isPrototypeOf()` will no longer be available.
- The object becomes a completely flat object with no access to the prototype chain.
- You won't be able to use many built-in JavaScript methods that rely on the prototype chain.

Why Is It Not Recommended?

- Disrupts expected object behavior.
- Prototypes are a powerful mechanism for sharing properties and methods between objects.

JavaScript OOP

There are 4 main principles in OOP:

- **Abstraction:** Hiding certain details that don't matter to the user and only showing essential features.
- **Encapsulation:** Keeping properties and methods private inside a class.
- **Inheritance:** Making all properties and methods available to a child class.
- **Polymorphism:** Having different forms. We can override a method inherited from a parent class.

Method Overriding

JS

```
class Animal {  
  speak() {  
    console.log("Generic animal sound");  
  }  
}  
  
class Dog extends Animal {  
  speak() {  
    console.log("Woof!");  
  }  
}  
  
const myDog = new Dog();  
myDog.speak(); // Woof!  
const myAnimal = new Animal();  
myAnimal.speak(); // Generic animal sound
```

Classes

A class is a template for creating objects that encapsulates data (properties) and behavior (methods).

Key Features of Classes

- **Constructor Method:** Special method for creating and initializing objects. Called automatically with `new`.
- **Methods:** Functions defined inside the class.
- **Inheritance:** Classes can extend other classes using the `extends` keyword.

Constructor Method

JS

```
class Car {
  brand = 'Unknown';
  model = 'Unknown';
  year = 0;

  constructor(brand, model, year) {
    this.brand = brand;
    this.model = model;
    this.year = year;
  }

  showDetails() {
    console.log(`Brand: ${this.brand}, Model: ${this.model}, Year: ${this.year}`);
  }
}

const myCar = new Car('Toyota', 'Corolla', 2020);
myCar.showDetails(); // Brand: Toyota, Model: Corolla, Year: 2020

const genericCar = new Car();
genericCar.showDetails(); // Brand: Unknown, Model: Unknown, Year: 0
```

`let`, `const`, or `var` are not used inside classes because you are defining properties that belong to each instance, not local variables.

Private Properties

Private properties use the `#` prefix to indicate they cannot be accessed outside the class.

JS

```
class BankAccount {
  #balance = 0;

  constructor(owner, initialBalance) {
    this.owner = owner;
    if (initialBalance) this.#balance = initialBalance;
  }

  deposit(amount) {
    this.#balance += amount;
  }

  getBalance() {
    return this.#balance;
  }
}
```

What Does "Private" Mean?

- Limited Access: Private properties can only be used inside the class where they were defined.
- Encapsulation: Private properties are not visible from outside the class.

JS

```
class BankAccount {
  #balance = 1000;

  constructor(owner) {
    this.owner = owner;
  }

  getBalance() {
    return this.#balance;
  }
}

const account = new BankAccount("Alice");
// console.log(account.#balance); // ERROR
console.log(account.balance); // undefined
console.log(account.getBalance()); // 1000
```

Getters and Setters

Getters and setters control how a property is accessed and modified.

```
class Person {
  #age;

  constructor(name, age) {
    this.name = name;
    this.#age = age;
  }

  get age() {
    return this.#age;
  }

  set age(newAge) {
    if (newAge > 0) {
      this.#age = newAge;
    } else {
      console.log('Age must be a positive number.');
```

Protected Properties

The underscore (`_`) prefix is a convention to indicate a property or method is "protected."

JS

```
class Animal {
  _name;

  constructor(name) {
    this._name = name;
  }

  _makeSound() {
    console.log(`${this._name} makes a sound`);
  }
}

class Dog extends Animal {
  constructor(name) {
    super(name);
  }

  bark() {
    this._makeSound();
    console.log('Woof!');
  }
}

const myDog = new Dog('Fido');
myDog.bark(); // Fido makes a sound, Woof!
console.log(myDog._name); // Fido
```

Inheritance with Class Syntax (ES6+)

JS

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  makeSound() {
    console.log(`${this.name} makes a sound`);
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name);
    this.breed = breed;
  }

  bark() {
    console.log('Woof!');
  }
}

const myDog = new Dog('Fido', 'Labrador');
myDog.makeSound(); // Fido makes a sound
myDog.bark(); // Woof!
console.log(myDog.breed); // Labrador
```

Static Members

Static properties and methods are associated with the class itself, not with instances.

JS

```
class MathUtils {
  static PI = 3.14159;

  static calculateCircumference(radius) {
    return 2 * this.PI * radius;
  }
}

console.log(MathUtils.PI); // 3.14159
console.log(MathUtils.calculateCircumference(5)); // 31.4159
```

Accessing Static Members: Static members are accessed through the class, not through instances.

JS

```
class Config {
  static apiUrl = 'https://api.example.com';

  static showUrl() {
    return this.apiUrl;
  }
}

const configInstance = new Config();
console.log(configInstance.apiUrl); // undefined
console.log(configInstance.showUrl()); // undefined
console.log(Config.apiUrl); // https://api.example.com
console.log(Config.showUrl()); // https://api.example.com
```

Inheritance and Static Members: Static members are inherited, but accessed through the class or subclass.

JS

```
class Vehicle {
  static type = 'Vehicle';

  static showType() {
    return this.type;
  }
}

class Car extends Vehicle {}

console.log(Vehicle.showType()); // Vehicle
console.log(Car.showType()); // Vehicle
```

Sets, Maps & Symbols

Sets

Los Set en JavaScript son una colección de valores únicos, lo que significa que un valor puede aparecer solo una vez en un Set.

Características

1. **Valores Únicos:** Un Set solo almacena valores únicos.
2. **Iteración:** Los valores se almacenan en el orden de inserción.
3. **No Indexado:** No puedes acceder a ellos usando un índice numérico.

Creación de un Set

JS

```
const miSet = new Set([1, 2, 3, 4, 5]);  
console.log(miSet); // Set(5) { 1, 2, 3, 4, 5 }
```

Métodos Comunes

add(value)

JS

```
miSet.add(6);  
miSet.add(2); // No se añade, ya que 2 ya existe en el Set  
console.log(miSet); // Set(6) { 1, 2, 3, 4, 5, 6 }
```

delete(value)

JS

```
miSet.delete(3);  
console.log(miSet); // Set(5) { 1, 2, 4, 5, 6 }
```

has(value)

JS

```
console.log(miSet.has(4)); // true
console.log(miSet.has(10)); // false
```

size

JS

```
const otroSet = new Set([1, 2, 3]);
console.log(otroSet.size); // 3
```

Iterando sobre un Set

for...of

JS

```
const numeros = new Set([10, 20, 30]);
for (const num of numeros) {
  console.log(num);
}
// 10
// 20
// 30
```

forEach()

JS

```
numeros.forEach((valor) => {
  console.log(valor);
});
// 10
// 20
// 30
```

How to Retrieve Data from a Set

Convert the whole Set to an array:

JS

```
const mySet = new Set([1, 2, 3]);
const myArray = Array.from(mySet); // or [...mySet]
console.log(myArray); // [1, 2, 3]
```

Extract specific values during iteration:

JS

```
let result;
for (const value of mySet) {
  if (value > 1) {
    result = value;
    break;
  }
}
console.log(result); // 2
```

Maps

Los Map en JavaScript son una colección de pares clave-valor donde tanto las claves como los valores pueden ser de cualquier tipo.

Tipos de Claves

String como Clave:

JS

```
const miMapa = new Map();
miMapa.set('nombre', 'Juan');
console.log(miMapa.get('nombre')); // 'Juan'
```

Número como Clave:

JS

```
miMapa.set(1, 'uno');
console.log(miMapa.get(1)); // 'uno'
```

Boolean como Clave:

JS

```
miMapa.set(true, 'verdadero');
miMapa.set(false, 'falso');
console.log(miMapa.get(true)); // 'verdadero'
console.log(miMapa.get(false)); // 'falso'
```

undefined y null como Claves:

JS

```
miMapa.set(undefined, 'indefinido');  
miMapa.set(null, 'nulo');  
console.log(miMapa.get(undefined)); // 'indefinido'  
console.log(miMapa.get(null)); // 'nulo'
```

Objeto como Clave:

JS

```
const objClave = { id: 1 };  
miMapa.set(objClave, 'objeto como clave');  
console.log(miMapa.get(objClave)); // 'objeto como clave'
```

Función como Clave:

JS

```
const funcionClave = function () { return 'Hola'; };  
miMapa.set(funcionClave, 'función como clave');  
console.log(miMapa.get(funcionClave)); // 'función como clave'
```

Array como Clave:

JS

```
const arrayClave = [1, 2, 3];  
miMapa.set(arrayClave, 'array como clave');  
console.log(miMapa.get(arrayClave)); // 'array como clave'
```

Características Principales

1. **Claves de Cualquier Tipo:** objetos, funciones, arrays, etc.
2. **Orden de Inserción:** Los pares se almacenan en el orden insertado.
3. **Tamaño Dinámico:** Propiedad `size`.

Creación de un Map

JS

```
const miMapa = new Map([  
  ['nombre', 'Juan'],  
  ['edad', 30],  
  [true, 'sí'],  
]);  
console.log(miMapa); // Map(3) { 'nombre' => 'Juan', 'edad' => 30, true => 'sí' }
```

Métodos Comunes

set(clave, valor)

JS

```
miMapa.set('ciudad', 'Madrid');  
miMapa.set('edad', 31); // Actualiza el valor de la clave 'edad'  
console.log(miMapa); // Map(4) { 'nombre' => 'Juan', 'edad' => 31, true =
```

get(clave)

JS

```
console.log(miMapa.get('nombre')); // Juan  
console.log(miMapa.get('país')); // undefined
```

has(clave)

JS

```
console.log(miMapa.has('edad')); // true  
console.log(miMapa.has('país')); // false
```

delete(clave)

JS

```
miMapa.delete('ciudad');  
console.log(miMapa); // Map(3) { 'nombre' => 'Juan', 'edad' => 31, true =
```

clear()

JS

```
miMapa.clear();  
console.log(miMapa); // Map(0) {}
```

size

JS

```
const nuevoMapa = new Map([[ 'a', 1 ], [ 'b', 2 ]]);  
console.log(nuevoMapa.size); // 2
```

Iterando sobre un Map

for...of

JS

```
for (const [clave, valor] of nuevoMapa) {  
    console.log(clave, valor);  
}  
// a 1  
// b 2
```

forEach()

JS

```
nuevoMapa.forEach((valor, clave) => {  
    console.log(clave, valor);  
});  
// a 1  
// b 2
```

keys(), values(), entries()

JS

```
const mapaEjemplo = new Map([  
    ['nombre', 'Ana'],  
    ['edad', 28]  
]);  
  
for (const clave of mapaEjemplo.keys()) {  
    console.log(clave); // 'nombre', 'edad'  
}  
for (const valor of mapaEjemplo.values()) {  
    console.log(valor); // 'Ana', 28  
}  
for (const [clave, valor] of mapaEjemplo.entries()) {  
    console.log(clave, valor); // 'nombre' 'Ana', 'edad' 28  
}
```

Symbols

Los Symbol en JavaScript son un tipo de dato primitivo que representan valores únicos e inmutables.

Características

1. **Unicidad:** Cada Symbol es único, incluso con la misma descripción.
2. **Inmutabilidad:** Una vez creado, no puede cambiar su valor.

Crear un Symbol

JS

```
const simbolo1 = Symbol();
const simbolo2 = Symbol('descripcion');
console.log(simbolo1); // Symbol()
console.log(simbolo2); // Symbol(descripcion)
```

Unicidad de los Symbol

JS

```
const simboloA = Symbol('id');
const simboloB = Symbol('id');
console.log(simboloA === simboloB); // false
```

Uso de Symbol como Claves de Propiedades

JS

```
const mySymbol = Symbol('property of object');
const myObject = {
  [mySymbol]: 'This is the value'
};
console.log(myObject[mySymbol]);
```

Colisión de Nombres con Strings

Cuando se usan strings como claves, si dos partes del código usan la misma clave, la segunda sobrescribe el valor.

JS

```
const objeto = {};
objeto['clave'] = 'valorA';
objeto['clave'] = 'valorB';
console.log(objeto['clave']); // 'valorB'
```

Cómo los Symbol Previenen Colisiones

Los Symbol son únicos y no colisionan, incluso con la misma descripción.

JS

```
const simboloA = Symbol('claveUnica');  
const simboloB = Symbol('claveUnica');  
  
const objeto = {};  
objeto[simboloA] = 'valorA';  
objeto[simboloB] = 'valorB';  
  
console.log(objeto[simboloA]); // 'valorA'  
console.log(objeto[simboloB]); // 'valorB'
```

Strings & String Methods

String y String Methods You can use single or double quotes:

JS

```
let carName1 = "Volvo XC60"; // Double quotes
let carName2 = 'Volvo XC60'; // Single quotes
```

Quotes Inside Quotes You can use quotes inside a string, as long as they don't match the quotes surrounding the string.

JS

```
let answer2 = "He is called 'Johnny'";
let answer3 = 'He is called "Johnny"';
```

Template Strings Templates are strings enclosed in backticks (`This is a template string`). Templates allow single and double quotes inside a string.

JS

```
let text = `He's often called "Johnny"`;
```

Escape Characters Because strings must be written within quotes, JavaScript will misunderstand this string:

JS

```
let text = "We are the so-called "Vikings" from the north.";
```

The string will be chopped to "We are the so-called ". To solve this problem, you can use a backslash escape character.

JS

```
let text = "We are the so-called \"Vikings\" from the north.";
let text= 'It\'s alright.';
let text = "The character \\ is called backslash.";
```

String methods

- JavaScript String Length The length property returns the length of a string.

JS

```
let text = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
let length = text.length;
console.log(length); // 26
```

- JavaScript String includes() The includes() method returns true if a string contains a specified string. Otherwise it returns false. The includes() method is case sensitive.

JS

```
let text = "Hello world, welcome to the universe.";
let result = text.includes("world");
console.log(result); // true
```

- JavaScript String trimStart(), trimEnd(), trim() trimStart() removes whitespace only from the start of a string. trimEnd() removes whitespace only from the end of a string. trim() removes whitespace from both sides of a string.

JS

```
let text1 = "    Hello World!    ";
let text2 = text1.trimStart();
let text3 = text1.trimEnd();
let text4 = text1.trim();

console.log(text1.length); // 22
console.log(text2.length); // 17
console.log(text3.length); // 17
console.log(text4.length); // 12
```

- JavaScript String replace() The replace() method replaces a specified value with another value in a string.

JS

```
let text = "Please visit Microsoft!";
let newText = text.replace("Microsoft", "W3Schools");
console.log(newText); // Please visit W3Schools!
```

- JavaScript String slice()

JS

`slice()` extracts a part of a string and returns the extracted part in a new

string. The method takes 2 parameters: start position, and end position (end not included).

JS

```
let text = "Apple, Banana, Kiwi";
let part = text.slice(7, 13);
console.log(part); // Banana
```

- JavaScript String `substring()` The `substring()` method of String values returns the part of this string from the start index up to and excluding the end index.

JS

```
let str = "Apple, Banana, Kiwi";
let part = str.substring(1,5);

console.log(part); // pple
```

If you omit the second parameter, `substring()` will slice out the rest of the string.

JS

```
let str = "Apple, Banana, Kiwi";
let part = str.substring(7);
console.log(part); // Banana, Kiwi
```

- JavaScript String `repeat()` The `repeat()` method returns a string with a number of copies of a string. The `repeat()` method returns a new string. The `repeat()` method does not change the original string.

JS

```
let text = "Hello world!";
let result = text.repeat(2);
console.log(result); // Hello world!Hello world!
```

- JavaScript String `split()` A string can be converted to an array with the `split()` method.

JS

```
let text = "a,b,c,d,e,f";
const myArray = text.split(",");
console.log(myArray); // [ 'a', 'b', 'c', 'd', 'e', 'f' ]
```

- Converting to Upper and Lower Case A string is converted to upper case with toUpperCase(): A string is converted to lower case with toLowerCase():

JS

```
let text1 = "Hello World!";

let text2 = text1.toUpperCase();
let text3 = text1.toLowerCase();
console.log(text2); // HELLO WORLD!
console.log(text3); // hello world!
```

String concatenation

String concatenation refers to combining multiple strings into a single string.

Using the + Operator

JS

```
let greeting = "Hello";
let name = "John";
let result = greeting + name;
console.log(result); // Output: HelloJohn
```

Using the += Operator

JS

```
let name = "John ";
name += "Doe";
console.log(name); // Output: John Doe
```

Using String concat() Method

JS

```
str1.concat(str2, str3, . . . , strN)
let str = 'Geeks';
let value = str.concat('for', 'Geeks');
console.log(value);
```

Using Template Literals (Template Strings)

JS

```
const producto = "Tablet de 12 pulgadas";
const precio = 30;
const marca = "Apple";
console.log(producto + ", $" + precio + " dolares, marca " + marca);
console.log(`${producto}, ${precio} dolares, marca ${marca}`);
```

Salida:

TEXT

```
Tablet de 12 pulgadas, $30 dolares, marca Apple
Tablet de 12 pulgadas, $30 dolares, marca Apple
```